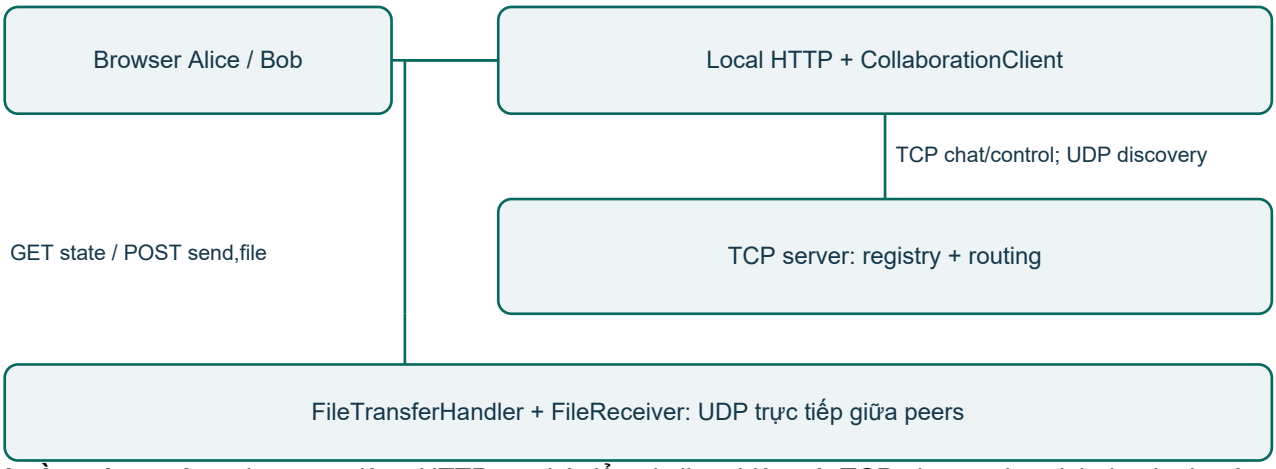
ASSIGNMENT 1

Distributed & Streaming-based Client-Server Collaboration

Báo cáo kỹ thuật và xác minh tích hợp

Role 4 - Client, UI & Testing | 05/11/2026

Baseline: 3c4174791a7e796a18e6d57c98c03111359cc1a5. Báo cáo mô tả implementation thực tế và bằng chứng kiểm thử, không xác nhận mọi yêu cầu đã đạt. Source Roles 1-3 không bị thay đổi bởi đợt finalization.



Luồng đang chạy: browser dùng HTTP cục bộ để gọi client hiện có. TCP phục vụ handshake, login và chat; UDP phục vụ heartbeat/discovery và file P2P. File được chia chunk 700 byte, gửi cửa sổ8, kiểm SHA-256 trước ghi đích. Nội dung chat và file dùng Fernet từ thư viện cryptography.

Ranh giới kiến trúc: static/index.html chỉ trình bày; client.py chứa HTTP bridge và socket client; server.py quản lý phiên và routing; file_client.py điều phối ACK/transfer; file_transfer.py mã hóa, serialization và reassembly. udp_streamer.py là module riêng, chưa được runtime này sử dụng.

Đề gốc: Assignment 1 PDF trang 1-4, SHA256 c16b856e…819f567. Guide slide7/18/22 nêu các điểm cần chốt về transport, congestion và HTTP UI. Không lấy starter demo làm chuẩn thay đề.

1. Protocol và phiên làm việc

Header chung TCP/UDP dùng network byte order, struct !HBBHH, dài8 byte. TCP đọc đủ header rồi đúng payload_length; UDP chứa một packet mỗi datagram. Header và routing metadata không được coi là encrypted.

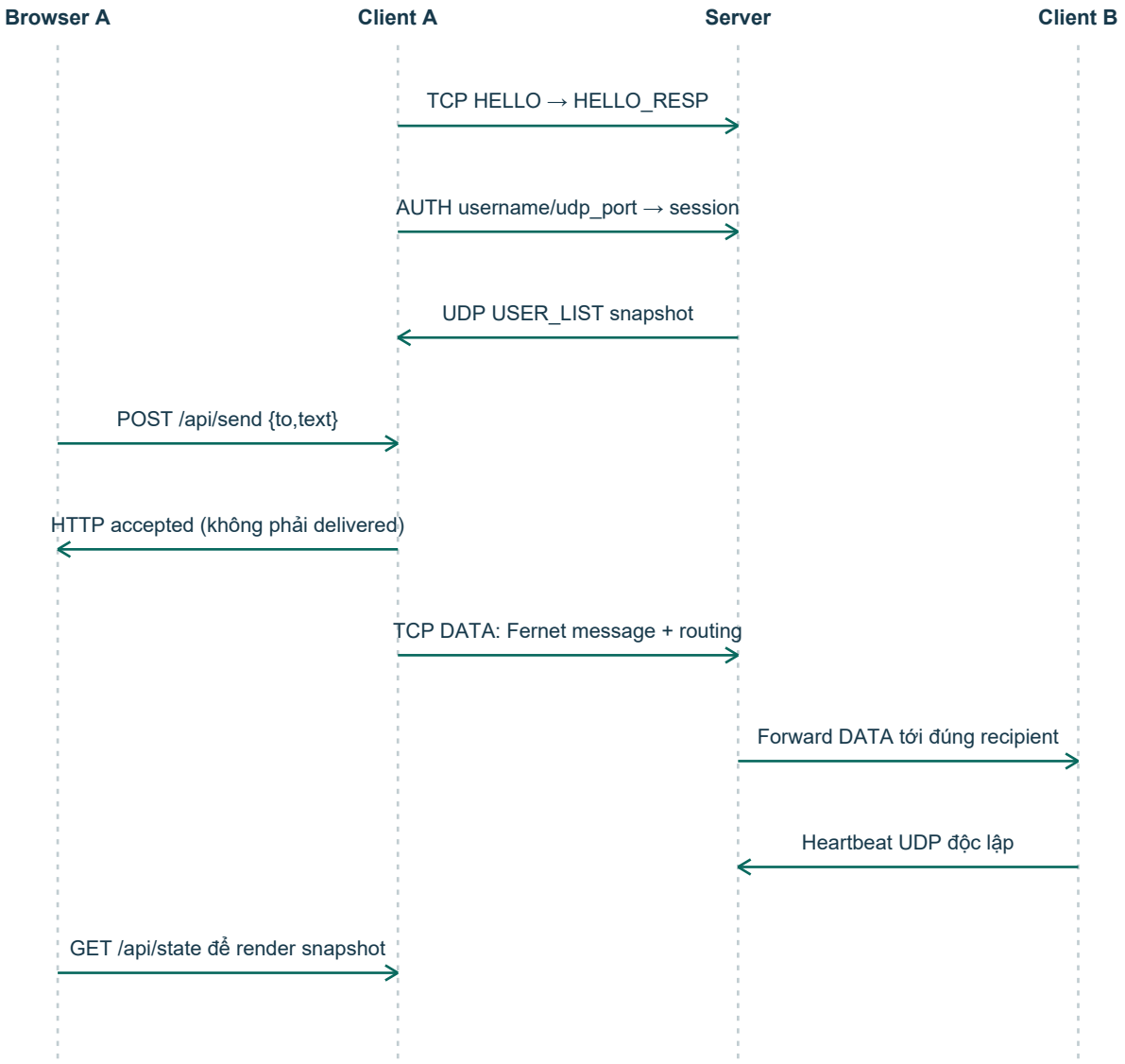
Trường	Độ dài	Ý nghĩa
Magic	2 byte	0xDCBA
Message type	1 byte	HELLO, AUTH, LOGOUT, heartbeat/user list, DATA/ACK/EOF
Status	1 byte	0:none, 1:OK, 2:error
Sequence	2 byte	0..65535; wrap cần semantics riêng theo từng luồng
Payload length	2 byte	0..65535 byte, không gồm header

Kênh / message	Payload và hành vi hiện tại
TCP HELLO/AUTH/LOGOUT	AUTH: username + udp_port; reply session_id. Không có password database. Server reject tên trùng và cổng ngoài1..65535.
TCP DATA	JSON mode/broadcast hoặc unicast, recipient, message chứa Fernet token, enc:true. Server thêm sender và forward; client decrypt message.
UDP heartbeat/discovery	Peer gửi heartbeat mỗi5s; server watchdog15s và quét3s. Snapshot user cóseq, client kiểm source/schema/order; server refresh định kỳ.
UDP DATA file	Encrypted JSON FILE_META, FILE_CHUNK, ACK và FILE_RESULT. file_id/chunk_index thuộc application payload. FILE_RESULT thông báo completed/failed.
Standalone streaming	MSG_EOF riêng sau fix; không còn dùng literal payload làm EOF. Vẫn cần chứng minh tích hợp và congestion control.

State phiên: START → TCP CONNECT → HELLO_OK → AUTH_OK → ACTIVE → LOGOUT/CLOSED. API state trả username, users, messages, transfers. HTTP thành công chỉ chứng minh bridge còn sống; chưa có verified server-liveness field. Tự nối lại phiên sau server restart chưa được triển khai.

Giới hạn security: Fernet key chung trong config chỉ phù hợp demo. Người biết key có thể decrypt/forge; không gọi đây là bảo mật danh tính end-to-end. Mã hóa chat mặc định không đồng nghĩa server bắt buộc mọi raw peer gửi ciphertext.

2. Sequence: login và message

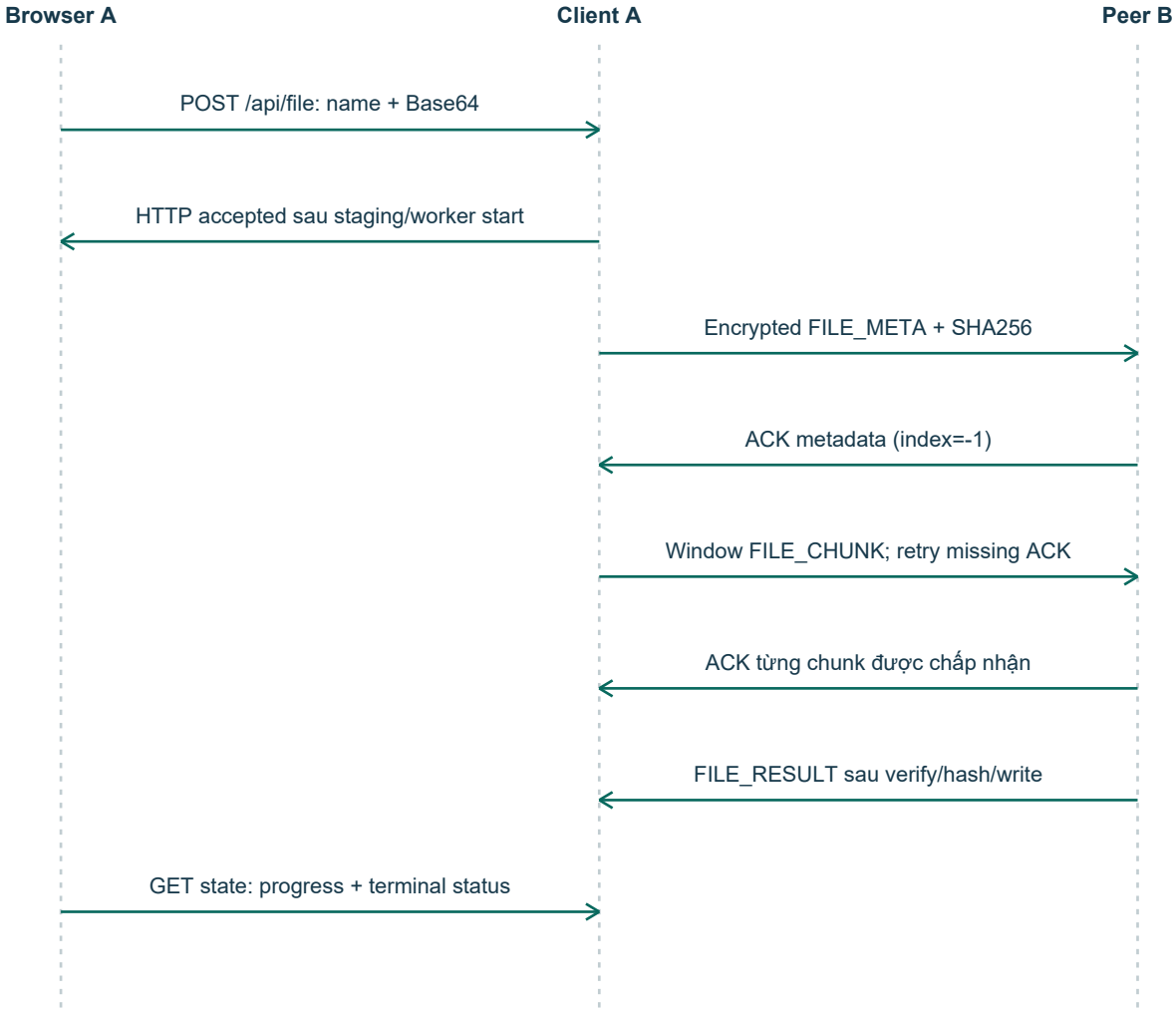


Client gửi lưu local message record sau sendall; không có ACK ứng dụng xác nhận người nhận đọc/hiển thị chat. Broadcast server gửi tới các peer khác; unicast gửi đúng username. Client B decrypt rồi thêm message vào state. UI không được đổi nhãn HTTP accepted thành delivered.

Concurrency: mỗi TCP session có thread; client có TCP receive, UDP receive, heartbeat và cleanup threads. Server dùng lock cho registry/send. Điều này hỗ trợ20 client đã kiểm, nhưng chưa chứng minh fairness/latency dưới slow-reader hoặc soak dài.

Ordering: TCP bảo đảm thứ tự byte trong một connection. UDP snapshot version giảm stale update; byte streaming async và loại bỏ HoL giữa logical streams chưa được chứng minh chỉ bằng việc có thread.

3. Sequence: file và completion



Receiver kiểm file size, chunk index, size/hash và đường dẫn. File đích tồn tại được từ chối để tránh overwrite. Sender mới dựa FILE_RESULT để biết completed/failed; tuy nhiên kết quả cuối vẫn phải đáng tin khi chính datagram result/ACK bị mất.

Hai ranh giới cần review: staging hiện dùng .upload_temp/basename nên request đồng thời trùng tên có thể thay bytes trước khi worker đọc. Sau đó SHA256 vẫn đúng đối với bytes đã bị thay, không còn đại diện file người gửi chọn. Ngoài ra trạng thái done phải ổn định sau worker kết thúc, không chỉ đúng tại một lần poll.

UI: upload 110MiB JSON tương đương khoảng82.5MiB raw file; layer file 100MiB. UI tính cả Base64/JSON overhead. Progress100 với status sending được ghi rõ 'Chờ kết quả từ bên nhận'; không giả completed. Draft/file được giữ khi HTTP operation thất bại.

4. Verification và khả năng tái lập

Lượt Role4	Total / pass / fail / error	Thời gian
initial_strict	21 / 20 / 0 / 1	148.105s
final_1	23 / 20 / 3 / 0	156.068s
final_2	23 / 20 / 3 / 0	158.522s

Các JSON/traceback gốc trong audits/finalization_2026_11_05. Initial suite có21 test; final suite bổ sung lost-result và same-name upload thành23. Không thêm skip/expectedFailure. Lỗi disk của fixture83MiB được sửa bằng browser File thật cùng size trong RAM; đây là test validation, không giả truyền83MiB qua mạng.

Kiểm chứng dương: server + client.main thật serve HTML, real HTTP chat/file, SHA256, progress;20 client đồng thời; kill client, heartbeat timeout, restart thủ công; browser malformed state/XSS/draft/API timeout/mobile/keyboard. Test backend/review bổ sung độc lập nằm trong các thư mục con tương ứng.

Bổ sung: file 24MiB hoàn tất55.653s, SHA-256 khớp và progress trung gian. Receiver timeout đã qua real HTTP và Edge DOM. Team file tests:29 pass / 6 fail do contract drift. Review phát hiện username '*' xung đột broadcast và client không có deadline đọc HELLO.

Test không đủ để suy ra: DOM fixture không chứng minh sockets; boolean sender không chứng minh receiver commit; completed tức thời không chứng minh terminal status ổn định. Test mới quan sát qua retry horizon và so bytes người gửi chọn với file nhận.

Chạy: python -m unittest discover -s tests -p "test_role4_*.py" -v. Demo có cleanup: python submission/assignment1/demo.py --smoke --output demo-result.json. Xem submission/assignment1/README.md để cài runtime/test dependencies và chạy3 terminal.

Chưa kiểm chứng: LAN nhiều máy, file sát100MiB/HTTP limit, diskfull có kiểm soát, prolonged soak, slow-reader fairness, BBR/CUBIC, simulator/OS khác. Disk gần đầy là môi trường giới hạn thật, không phải lý do bỏ invariant hay che failure.

5. Challenges & Solutions

Trang reflection này tổng hợp bài học có bằng chứng từ implementation, Git và phép thử. Nhóm cần duyệt nội dung và bổ sung trải nghiệm cá nhân trước khi nộp; không tự gán ai đã làm công việc chưa được xác nhận.

Độ tin cậy có nhiều tầng. HTTP200 chỉ xác nhận bridge nhận yêu cầu; sendall chỉ gửi bytes cho TCP; chunk ACK chỉ xác nhận một bước trong file transfer. Trước đây sender có thể báo done khi receiver từ chối file. Upstream thêm FILE_RESULT để tách kết quả receiver khỏi ACK chunk. Bài học là cần đặc tả terminal outcome rõ ràng, bounded retry cho completion, và kiểm tra cả hai phía cùng nội dung trên đĩa.

Race xảy ra trước khi network protocol bắt đầu. Hai upload cùng basename ghi chung staging có thể làm worker đầu đọc file của request sau. Mã hóa và checksum không giải quyết thay thế trước bước tính hash. Giải pháp bàn giao là staging riêng cho từng request, giữ tên gốc trong metadata và cleanup theo lifetime worker; Role4 giữ regression test thay vì sửa file protected.

Timeout phải phản ánh hoạt động. Upstream chuyển từ created_at sang last_activity để file đang tiến triển không bị coi như abandoned chỉ vì chạy lâu. Đây là thay đổi có ý nghĩa với file lớn. Tuy vậy, timeout nhận data và timeout chờ terminal result là hai state khác nhau; test mất FILE_RESULT cần thiết ngay cả khi24MiB không loss có thể hoàn tất.

Discovery chịu ảnh hưởng datagram loss/order. Source/schema validation, seq và refresh giúp tránh dữ liệu cũ hoặc không hợp lệ làm hỏng peer map. Test phải kiểm convergence sau mất update cuối, không chỉ danh sách đúng ngay sau một login. Wraparound, server restart và source authentication mạnh hơn vẫn cần threat model/acceptance rõ.

Test xanh có thể trả lời sai câu hỏi. Test cũ mất ACK cuối dừng khi vừa thấy done; worker có thể còn retry và ghi đè thành failed. Role4 đổi test để đợi chính worker kết thúc, xác minh fault thực sự xảy ra và byte-exact. Fixture file lớn từng làm hết disk; browser File cùngsize kiểm cùng validation mà không cần giả kích thước hoặc giảm yêu cầu.

Tách trách nhiệm giúp review được. Browser không tự tạo ACK/retry/crypto mới. Mọi thay đổi ở đây nằm trong UI, test, demo và tài liệu Role4; network fixes được bàn giao với reproduction. Bài học cuối là ghi rõ điều chưa đạt trước khi bảo vệ: tích hợp streaming và congestion không được suy ra từ tên module hoặc số lượng test pass.

6. Readiness và bàn giao

Phạm vi	Kết luận và việc cần làm
Demo cơ bản	Có thể trình diễn login/users/chat/file nhỏ và browser UI; dùng evidence thực. Không coi demo này là phê duyệt toàn đề.
Toàn vẹn upload	Client owner cần staging per request; cùngbasename phải giữ nguyên bytes của từng người nhận.
File terminal reliability	Role3 cần completion-result retry/ACK, terminal status bất biến và xử lý mất final ACK. Không được bỏ qua regression mới.
Streaming/congestion	Role1/2 cần chốt transport với giảng viên, chứng minh runtime dùng byte stream async và mức BBR/CUBIC yêu cầu.
Submission docs	PDF này có architecture/protocol/sequence/reflection. README riêng trong gói cung cấp startup; shared README vẫn cần owner duyệt cập nhật.
Môi trường và giới hạn	Đĩa gần đầy; không tự xóa dữ liệu người dùng hoặc cài simulator nặng. Giải phóng/chuyển dữ liệu có chủ đích trước thử lớn hơn.

Identity/startup: server chấp nhận username '*' nhưng client dùng '*' cho broadcast, khiến thông điệp định gửi riêng có thể tới người khác. TCP connect timeout không giới hạn HELLO/AUTH vì socket trở lại blocking trước khi đọc. Hai vấn đề có reproduction và handoff riêng.

Verdict: chưa đủ bằng chứng để tuyên bố Assignment1 hoàn thành toàn bộ. Các lỗi còn lại có test/handoff riêng, không được biến thành expectedFailure để nộp suite xanh. Báo cáo cuối Markdown chứa status chi tiết của từng yêu cầu và kết quả backend/review mới nhất.

Thứ tự tiếp theo: sửa integrity/terminal protocol trước; giữ UI hiển thị trung thực; rerun regression và fault tests; hoàn thiện phạm vi streaming/congestion; xác minh LAN và hồ sơ nộp. Assignment2 là dự án thiết kế hạ tầng riêng; chỉ tái dùng phương pháp evidence và workload A1 khi có lý do cụ thể, không dùng A1 thay topology/configurations.

Nguồn đối chiếu: official Assignment1 PDF trang 1-4; Assignment1_Network_Application_Guide slide7,18,22,27; current protocol.py/client.py/server.py/file_client.py/file_transfer.py/udp_streamer.py; Git baseline3c41747; logs/JSON tại audits/finalization_2026_11_05. Tài liệu protocol cũ có thể lệch runtime; bảng ở báo cáo này mô tả code đã đọc.